Question 7: Create a Jenkinsfile defining pipeline stages (build, test, deploy) and demonstrate a declarative or scripted pipeline execution.

Executed on Ubuntu 24.04 LTS (WSL2) by Meenakshi Rakesh. Repository: https://github.com/meenakshi16rp/Devops-lab-1-

1. The Jenkinsfile (committed at the repository root)

Jenkinsfile

```
// ENSP461 Lab 1 - Declarative CI/CD pipeline
// Stages: Build -> Test -> Deploy, triggered from GitHub
pipeline {
    agent any

    environment {
        VENV = "$WORKSPACE/.venv"
        DEPLOY_DIR = "$HOME/enp461-deploy"
    }

    stages {
        stage("Checkout") {
            steps {
                echo "Pulling source code from GitHub repository..."
                checkout scm
            }
        }

        stage("Build") {
            steps {
                echo "Building application: creating virtual environment and installing \
      dependencies..."
                sh """
                    python3 -m venv $VENV
                    $VENV/bin/pip install --quiet -r app/requirements.txt
                """
            }
        }

        stage("Test") {
            steps {
                echo "Running unit tests with pytest..."
                sh "$VENV/bin/python -m pytest app/tests -v"
            }
        }

        stage("Deploy") {
            steps {
                echo "Deploying application to local deployment directory..."
                sh "./deploy/deploy.sh"
            }
        }
    }

    post {
        success {
            echo "Pipeline SUCCESS: application built, tested and deployed."
        }
        failure {
            echo "Pipeline FAILED - check the stage logs."
        }
        always {
            echo "Cleaning workspace temporary files..."
            cleanWs()
        }
    }
}
```

2. Declarative pipeline structure

- pipeline { agent any } - declarative syntax; runs on any available executor.
- environment block defines VENV (build virtualenv) and DEPLOY_DIR once for all stages.
- stages: Checkout (checkout scm) -> Build (venv + pip install) -> Test (pytest -v) -> Deploy (deploy.sh with health verification).
- post section reports success/failure and always cleans the workspace.
- echo steps make each stage's intent visible in the console log - useful for viva demonstration.

3. Execution evidence

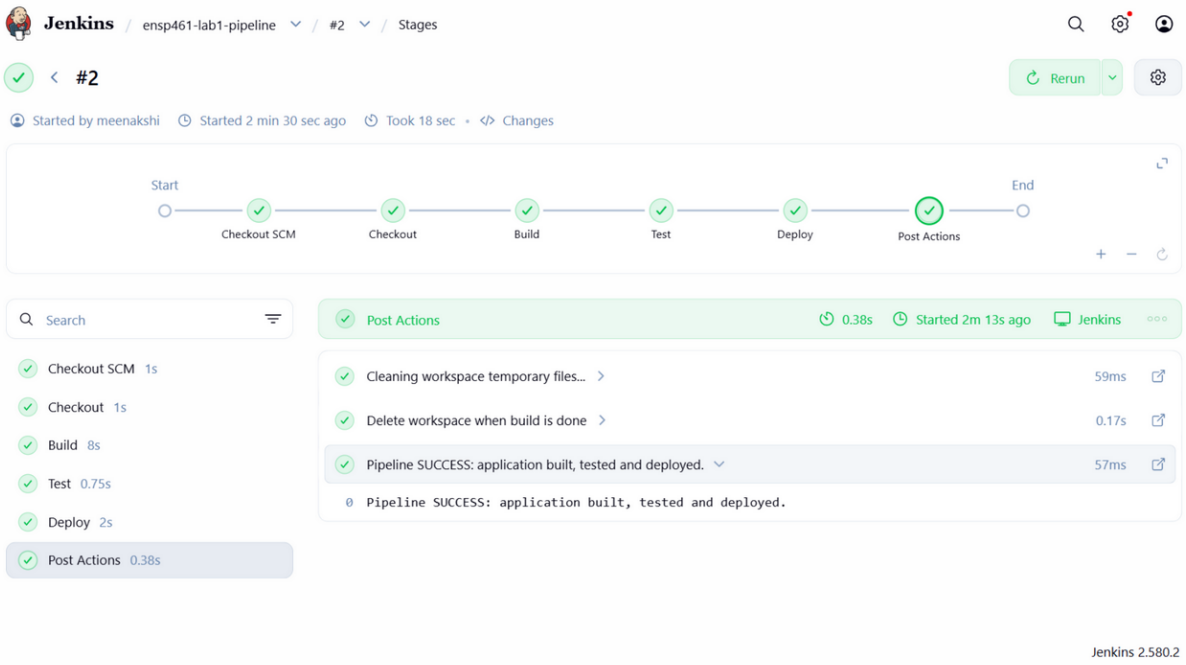


Figure: Stage graph: Checkout SCM, Checkout, Build, Test, Deploy, Post Actions - all successful

Stage	Duration (build #2)	Key step
Checkout SCM	1 s	git fetch from GitHub
Checkout	1 s	checkout scm
Build	8 s	python3 -m venv + pip install
Test	0.75 s	pytest: 3 passed
Deploy	2 s	deploy.sh + health check OK
Post Actions	0.38 s	success echo + cleanWs()

console excerpt: test summary, deploy verification, final status

```
============================== 3 passed in 0.19s ===============================
[Pipeline] echo
Deploying application to local deployment directory...
Deployment verified: health OK
Pipeline SUCCESS: application built, tested and deployed.
Finished: SUCCESS
```

4. Observations

- Declarative pipelines give a fixed, readable structure (stages/steps/post) that maps directly onto the stage graph UI.
- Keeping the Jenkinsfile in the repository means every pipeline change is itself reviewed, versioned and built - CI for the CI definition.
- Stage timings show the build is dominated by dependency installation; caching could shorten it further.